day4-spark

Day 4: Apache Spark, Knowledge Review Answers

Answers to the 15 review questions at the end of the Day 4 slides (Spark architecture, RDDs, DataFrames, Spark SQL, optimization, MLlib and GraphX).

Conceptual questions

Q1. Explain why Apache Spark was developed. What specific limitations of disk-oriented batch processing did it address?

Spark was built to make **iterative** and multi-step jobs fast. MapReduce writes every intermediate result to disk and starts a new job for each step, so a machine learning algorithm that loops 50 times reads and writes the same data 50 times.

Spark addressed three limits:

- **Repeated disk I/O.** Spark can keep a dataset in memory across steps (`cache()`), so each loop reads RAM, not disk.
- **Job overhead and chaining.** In MapReduce the programmer chains many jobs by hand. In Spark one program describes the whole pipeline and the engine splits it into stages.
- **Limited programming model.** MapReduce only offers map and reduce. Spark adds joins, filters, SQL, machine learning and graph processing in one engine.

Fault tolerance comes from lineage (recompute lost parts) instead of writing every step to replicated storage.

Q2. Distinguish the driver, executors, and the cluster manager. What happens if the driver dies? What happens if an executor dies?

Part	Role
Driver	Runs your `main` program, builds the DAG, splits it into stages and tasks, schedules tasks, collects results
Executors	Processes on worker nodes that run tasks and hold cached data and shuffle files
Cluster manager	Gives out CPU and memory (YARN, Kubernetes, Standalone, or local mode)

- **Driver dies:** the whole application fails. All its state (the DAG, task tracking, cached data references) is lost. You must restart the job. (In YARN cluster mode the ApplicationMaster can retry the driver, but it starts from the beginning.)
- **Executor dies:** the application survives. The driver reruns the lost tasks on other executors, and recomputes any lost cached or shuffle data from lineage. The job is slower but finishes.

Q3. Explain the difference between transformations and actions. Why does Spark defer execution until an action is called?

A **transformation** (`map`, `filter`, `join`, `groupBy`) returns a new RDD or DataFrame and only records the step. An action (`count`, `collect`, `show`, `write`) returns a result or writes data, and it is the only thing that starts a job.

Spark waits (lazy evaluation) because seeing the whole chain first lets it:

- fuse many narrow steps into one stage, so data is read once;
- push filters and column selection down to the data source;
- skip work whose result is never used.

```
df = spark.read.parquet("sales")          # nothing runs
big = df.filter(df.amount > 500)          # nothing runs
big.count()                               # job starts here
```

Q4. Distinguish narrow from wide transformations. Give an example of each and explain why the difference matters.

- **Narrow:** each output partition needs data from only one input partition. Examples: `map`, `filter`, `flatMap`, `withColumn`. No data moves between machines.
- **Wide:** an output partition needs data from many input partitions. Examples: `groupByKey`, `reduceByKey`, `join`, `distinct`, `repartition`. Data must be **shuffled** across the network.

Why it matters: every wide transformation ends a stage. A shuffle writes data to disk, sends it over the network and waits for all upstream tasks, so it is usually the most expensive part of a job. Fewer and smaller shuffles mean faster jobs. In `explain()` output a shuffle shows up as `Exchange`.

Q5. Explain how Spark recovers from a lost partition without replicating intermediate data.

Through **lineage**. Every RDD remembers its parent and the function that built it. If a partition is lost (executor crash), Spark walks back through the lineage to the nearest data that still exists (the source file in HDFS, a cached partition, or surviving shuffle files) and reruns only the steps needed for that one partition.

- Narrow dependencies: recompute just the matching parent partition. Cheap.
- Wide dependencies: may need the shuffle output of many parent partitions; if those shuffle files are also lost, the earlier stage reruns.

When lineage gets very long (iterative algorithms), use `checkpoint()` to save the data to reliable storage and cut the lineage.

Comparative questions

Q6. Compare RDDs, DataFrames, and Datasets across schema, type safety, optimization, and language support.

	RDD	DataFrame	Dataset
Schema	None (opaque objects)	Yes (named, typed columns)	Yes
Type safety	Compile-time for the objects (Scala/Java)	Checked at runtime (column names are strings)	Compile-time (Scala/Java)
Optimization	None by Catalyst	**Catalyst** + Tungsten	Catalyst + Tungsten
Languages	Scala, Java, Python	Scala, Java, Python, R, SQL	Scala, Java only
Use it for	Custom low-level logic, unstructured data	Most analytics and ETL	Typed Scala/Java code

A DataFrame is a `Dataset[Row]` in Scala. In Python you only have RDDs and DataFrames.

Q7. Explain the role of the Catalyst optimizer. What rewrites does it apply, and what does it need in order to apply them?

Catalyst turns a DataFrame or SQL query into an efficient physical plan. It goes through four phases: analysis (resolve names and types against the schema or catalog), logical optimization (rule-based rewrites), physical planning (pick join and aggregation strategies, using cost estimates), and code generation (Tungsten whole-stage codegen).

Common rewrites:

- predicate pushdown (filter before join, or inside the Parquet reader);

- column pruning (read only needed columns);
- constant folding (`1 + 2` becomes `3`);
- choosing broadcast join instead of sort-merge join when one side is small;
- combining adjacent filters and projections.

What it needs: a **schema** and declarative operations it can see inside. It cannot optimize what it cannot read, so an RDD lambda or a Python UDF is a black box to it.

Q8. Compare broadcast joins and shuffle joins. When is each appropriate?

	Broadcast hash join	Shuffle join (sort-merge or shuffle hash)
How	Small table is copied to every executor; big table stays where it is	Both tables are shuffled so equal keys meet in the same partition
Shuffle of the big table	None	Yes
Memory	Small side must fit in each executor	Works for any size, spills to disk
Use when	One side is small (lookup or dimension table)	Both sides are large

Spark broadcasts automatically when a side is below `spark.sql.autoBroadcastJoinThreshold` (10 MB by default). You can force it with a hint:

```
from pyspark.sql.functions import broadcast
sales.join(broadcast(products), "product_id")
```

Broadcasting a table that is not really small can cause out-of-memory errors on the driver or executors.

Q9. Explain the difference between groupByKey and reduceByKey. Which is preferred, and why?

`groupByKey` shuffles **every value** across the network and then builds a list per key. `reduceByKey` first combines values inside each partition (a map-side combine, like a MapReduce combiner) and only shuffles one partial result per key per partition.

`reduceByKey` is preferred for aggregations: much less shuffle data, less memory, and no risk of one key's huge list not fitting in memory. Use `groupByKey` only when you really need all the values (for example, to sort them).

```
pairs = words.map(lambda w: (w, 1))
counts = pairs.reduceByKey(lambda a, b: a + b)          # preferred
counts2 = pairs.groupByKey().mapValues(len)             # same result, more shuffle
```

The function given to `reduceByKey` must be associative and commutative (sum, max, min), because Spark applies it in any order.

Q10. Distinguish the DataFrame-based spark.ml API from the older RDD-based spark.mllib API. Which is the primary API for new projects?

- `spark.ml`: works on DataFrames, uses Transformers, Estimators and Pipelines, benefits from Catalyst, gets all new features. This is the **primary API**.
- `spark.mllib`: works on RDDs of vectors and `LabeledPoint`. It has been in maintenance mode since Spark 2.0 (bug fixes only).

New projects should use `spark.ml`. "MLlib" is still the name of the library as a whole.

Scenario-based questions

Q11. A Spark job takes two hours, and one task in a shuffle stage runs ten times longer than the others. Diagnose the problem and propose two mitigations.

Diagnosis: **data skew**. One key (often a very common value, or `NULL`) holds far more rows than the others, so the partition that receives it does most of the work while the other tasks finish and wait. Confirm in the Spark UI: open the stage, look at the task summary; max duration and shuffle read size are far above the median.
Mitigations:

1. **Salting:** add a random number to the hot key so its rows spread over many partitions, aggregate, then aggregate again without the salt.

```
from pyspark.sql import functions as F
N = 8
salted = df.withColumn("salt", (F.rand() * N).cast("int"))
partial = salted.groupBy("key", "salt").agg(F.sum("amount").alias("s"))
result = partial.groupBy("key").agg(F.sum("s").alias("total"))
```

2. **Adaptive Query Execution skew join:** with `spark.sql.adaptive.enabled` and `spark.sql.adaptive.skewJoin.enabled` (both on by default in Spark 3.2+), Spark splits oversized join partitions at runtime.
3. Other options: broadcast the smaller side if possible, filter out or handle `NULL` keys separately, or process the few hot keys in a separate job and `union` the results.

Q12. You join two very large DataFrames and observe a heavy shuffle. What do you check first, and what changes might reduce the shuffle cost?

Check first:

- Is one side really large, or could it be broadcast after filtering?
- Are filters and column selection applied before the join? (Look at `explain()`: are the filters below the `Exchange`?)
- Is the shuffle balanced, or is one key skewed (Spark UI task sizes)?
- Do the join key types match on both sides (a string key joined to an int adds casts and can block optimization)?
- Is `spark.sql.shuffle.partitions` (default 200) sensible for the data size?

Changes that reduce shuffle cost:

- filter early and `select` only the needed columns;
- pre-aggregate one side before joining if the result only needs totals;
- broadcast a side that becomes small after filtering;
- if the same large tables are joined again and again, write them **bucketed** on the join key (`bucketBy`) so later joins skip the shuffle;
- let AQE coalesce small partitions and handle skew.

Q13. A team trains a model on data that fits comfortably on one machine. Would Spark be the right choice? Explain.

Usually no. On one machine, tools like pandas, scikit-learn, XGBoost or PyTorch are faster to run, faster to iterate with, and have more algorithms. Spark adds overhead: job scheduling, serialization between JVM and Python, and network coordination, with no benefit when the data already fits in RAM.
Spark becomes the right choice when:

- the data will grow beyond one machine soon;

- the features are already produced by a Spark pipeline and moving data out is costly;
- you need to run many trainings in parallel (for example, a large hyperparameter search).

Q14. An analyst wants sub-second response time on a 10 GB dataset. Is Spark appropriate? What alternatives would you recommend?

Not a good fit. Spark is built for throughput, not latency. Each query pays for planning, job scheduling and task launch, which is often hundreds of milliseconds to several seconds even on cached data.

Better options:

- **DuckDB** or a single-node columnar engine: 10 GB fits on one good machine and queries return in milliseconds.
- An OLAP store built for interactive queries: ClickHouse, Apache Druid, Apache Pinot.
- A distributed SQL engine for ad hoc queries over a data lake: Trino or Presto.
- Use Spark to precompute aggregates on a schedule and serve the small results from a database or cache.

Q15. You must analyze a social graph with 500 million users and 5 billion edges. Which Spark library would you use, and what design considerations would guide the implementation?

Use **GraphX** (Scala or Java), or GraphFrames for a DataFrame and Python API. Both offer PageRank, connected components and triangle counting. GraphX itself gets few new features now, so check the Spark version you run.

Design considerations:

- **Partitioning:** choose a good edge partition strategy (for example `EdgePartition2D`) to cut communication.
- **High-degree vertices:** celebrity accounts with millions of followers cause skew; consider special handling or sampling.
- **Memory:** 5 billion edges need a large cluster; store graph data as Parquet and size executors for it.
- **Iterations:** each Pregel superstep is a full distributed round; set a max number of iterations or a convergence tolerance.
- **Checkpointing:** checkpoint every few iterations to keep lineage short.
- **Join back:** results (rank, component id) are joined back to user attributes with Spark SQL for reporting.

How to read `explain()` output

Run `df.explain()` (or `df.explain(True)` for all plan levels) and read the physical plan from the bottom up.

You see	It means
`FileScan parquet`	Reading the source files
`ReadSchema: struct<a,b>`	Only these columns are read (column pruning worked)
`PushedFilters: [GreaterThan(amount,500)]`	Filter sent into the file reader; Parquet/ORC can skip row groups
`PartitionFilters: [dt = 2024-03-15]`	Whole partition folders skipped (partition pruning)
`Exchange hashpartitioning(key, 200)`	A **shuffle**: a stage boundary
`BroadcastExchange` + `BroadcastHashJoin`	Small side copied to all executors; big side not shuffled
`SortMergeJoin`	Both sides shuffled and sorted; normal for two big tables
`HashAggregate(partial_...)` then `Exchange` then `HashAggregate`	Map-side partial aggregation, then final aggregation
`InMemoryTableScan`	Reading from a cached DataFrame
`AdaptiveSparkPlan isFinalPlan=false`	AQE is on; the final plan is decided at runtime

Caching example:

```
df = spark.read.parquet("sales").filter("amount > 500").cache()
df.count()                        # first action fills the cache
df.groupBy("region").count().show()
df.explain()                      # shows InMemoryTableScan
df.unpersist()                    # free the memory when done
```

Quick recall

- Spark is a processing engine; it does not store data.
- One action = one job. Each shuffle splits a job into another stage. One task per partition per stage.
- Driver death kills the app; executor death only reruns tasks.
- Lineage replaces replication for intermediate data; `checkpoint()` cuts long lineage.
- Narrow: `map`, `filter`. Wide (shuffle): `groupByKey`, `reduceByKey`, `join`, `distinct`, `repartition`.
- Prefer `reduceByKey` over `groupByKey` for aggregations.
- DataFrames and SQL use Catalyst and Tungsten; RDD lambdas and Python UDFs are black boxes.
- Broadcast threshold default: 10 MB (`spark.sql.autoBroadcastJoinThreshold`).
- Default shuffle partitions for DataFrames: 200 (`spark.sql.shuffle.partitions`).
- Skew: one slow task in a stage. Fix with salting, AQE skew join, or broadcast.
- Cache only data used more than once; `unpersist()` when done.
- `spark.ml` (DataFrames) is the current ML API; `spark.mllib` (RDDs) is maintenance only.